FinBERT: Fine-Tuning BERT for Downstream Financial Sentiment Analysis

James Phelan, Tony Zhao, Nathan Kwei, Bilal Ali

May 2025

1 Introduction

We are implementing FinBERT [1], a financial-domain variant of BERT [2], for sentiment analysis on financial text. FinBERT's original authors demonstrated that fine-tuning a large pre-trained language model on a modestly sized, domain-specific corpus can yield high accuracy in classifying financial statements and news headlines despite scarcity of labeled data.

We chose to build upon FinBERT because we wanted hands-on experience with fine-tuning transformer-based language models on specialized NLP tasks.

2 Methodology

2.1 Data Sources and Preprocessing

We conduct experiments on the Financial PhraseBank dataset [5], which comprises English sentences from financial news annotated with positive, negative or neutral sentiment. When `hf_dataset_name` and `hf_config_name` are provided, data are loaded directly via the HuggingFace `load_dataset()` API. Otherwise, tab-separated CSV files (`train.csv`, `validation.csv`, `test.csv`) are read using `pandas`.

- **Text encoding:** All sentences are tokenized with the `BertTokenizerFast` (uncased vocabulary) and padded or truncated to a fixed maximum length (`max_seq_length`).
- **Batches:** TF Datasets are created from token IDs and corresponding integer labels, shuffled (buffer size 10 000) for training, and batched (`train_batch_size` and `eval_batch_size`).

2.2 Model Architecture

We fine-tune a pre-trained BERT base model [2] for sequence classification ("bert-base-uncased") by appending a single linear classifier head with $|\mathcal{L}| = 3$

outputs. All encoder layers and the new classification head are exposed to gradient updates through gradual unfreezing (see2.3.2).

2.3 Training Procedure

2.3.1 Optimizer and Learning Rate Schedule

We employ AdamW optimization [4] with a slanted triangular learning-rate schedule [3]. Let T be the total number of training steps (steps_per_epoch $\times$ num_train_epochs), and γ the warmup proportion. The learning rate at step t is

$$\eta_t = \begin{cases} \eta_{\max} \frac{t}{\gamma T}, & t \leq \gamma T, \\ \eta_{\max} \frac{1 - t/T}{1 - \gamma}, & t > \gamma T. \end{cases}$$

Here, $\eta_{\max}$ =`learning_rate` and γ =`warmup_proportion`. This schedule enables rapid warmup and a gradual decay.

2.3.2 Gradual Unfreezing and Discriminative Fine-Tuning

To stabilize fine-tuning and leverage hierarchical feature representations, we freeze all BERT encoder layers at initialization. During each epoch, we "unfreeze" one additional encoder layer from top to bottom. Concretely:

1. Freeze $\{\ell_0, \ldots, \ell_{L-1}\}$ and train only classifier head.
2. After each epoch e, unfreeze encoder layer ℓ_{L-e}.

This gradual unfreezing scheme prevents catastrophic forgetting in lower layers and enables discriminative fine-tuning by applying distinct learning rates per layer group.

2.3.3 Handling Class Imbalance

We compute class weights inversely proportional to the square root of class frequencies in the training split:

$$w_c \propto \frac{1}{\sqrt{n_c}},$$

where n_c is the number of examples with label c. These weights are injected into a weighted cross-entropy loss.

2.4 Cross-Validation and Evaluation

We assess model generalization via stratified K-fold cross-validation ($K = 10$), shuffling with a fixed random seed. For each fold:

1. Split data into training (64%), validation (16%), and test (20%) sets.

2. Fine-tune the model following the procedure above.

3. Compute accuracy, macro-averaged precision, recall, and F1 on the held-out fold.

We report per-fold metrics as well as their mean and standard deviation across folds.

2.5 Implementation Details

All experiments are implemented in TensorFlow2.0 and HuggingFace Transformers. Key hyperparameters (e.g. batch sizes, learning rates, sequence length, number of epochs) are specified in a central `Config` class.

3 Results

Table 1: 10-fold Cross-Validation Results

Fold	Accuracy	Precision	Recall	F1-score
1	0.8062	0.8303	0.6157	0.5954
2	0.8106	0.7577	0.7016	0.7231
3	0.7930	0.4905	0.5832	0.5313
4	0.7753	0.6871	0.6056	0.6089
5	0.8009	0.7068	0.5932	0.5886
6	0.8230	0.8346	0.6892	0.7277
7	0.7478	0.8075	0.5632	0.5304
8	0.8230	0.8449	0.6695	0.6632
9	0.8363	0.8216	0.6637	0.6990
10	0.7566	0.7072	0.5742	0.5829
Mean ± SD	**0.7973 ± 0.0278**	**0.7488 ± 0.1030**	**0.6259 ± 0.0480**	**0.6251 ± 0.0701**

3.1 Cross-Validation Performance

Table 1 reports the results of our 10-fold cross-validation. We observe that the model achieves a mean accuracy of 0.7973 ± 0.0278, mean precision of 0.7488 ± 0.1030, mean recall of 0.6259 ± 0.0480, and mean F1-score of 0.6251 ± 0.0701.

Figure 1 visualizes these metrics per fold, illustrating that while accuracy remains relatively stable across folds, recall and F1-score exhibit greater variability, particularly in Fold 3 where precision dips markedly.

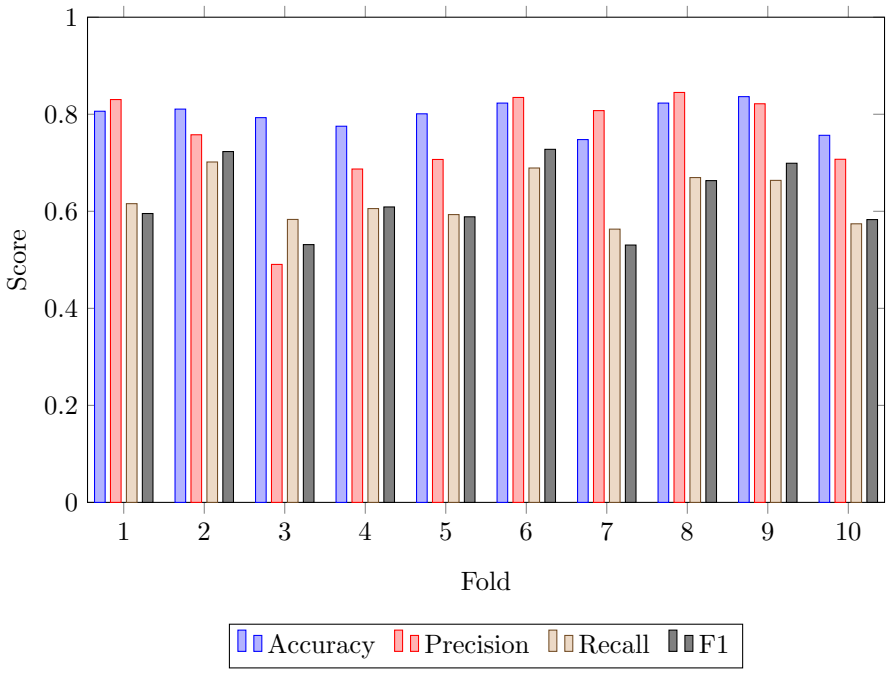

Figure 1: Per-fold cross-validation metrics (Accuracy, Precision, Recall, F1) for each of the 10 folds.

3.2 Effect of Gradual Unfreezing

To assess the impact of unfreezing increasing numbers of BERT encoder layers, we report in Table 2 the validation loss and accuracy when unfreezing 3, 6, 9, and all 12 layers. Both validation loss and accuracy improve as more layers are made trainable, with the lowest mean validation loss (0.877 ± 0.058) and highest mean validation accuracy (0.797 ± 0.028) achieved when all 12 layers are unfrozen.

Unfrozen Layers	Val Loss	Val Accuracy
3	1.280 ± 0.056	0.724 ± 0.020
6	1.005 ± 0.056	0.765 ± 0.024
9	0.910 ± 0.059	0.789 ± 0.026
12	0.877 ± 0.058	0.797 ± 0.028

Table 2: Validation loss and accuracy versus number of unfrozen BERT encoder layers

3.2.1 Loss Dynamics

Figure 2 plots the training and validation loss as a function of the number of unfrozen layers. As expected, training loss monotonically decreases from 1.45 (1 layer) to 0.76 (12 layers), indicating greater model capacity. Validation loss similarly declines from 1.24 to 0.87, demonstrating that unfreezing more layers yields better generalization on the validation split.

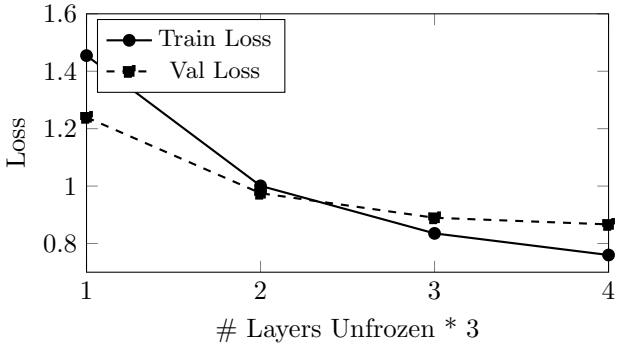

Figure 2: Training vs. validation loss as we gradually unfreeze 1–12 BERT layers.

3.2.2 Accuracy Dynamics

Figure 3 shows the corresponding training and validation accuracy curves. Training accuracy increases from 0.63 to 0.81 as layers are unfrozen, while validation accuracy improves from 0.75 to 0.81, peaking at 9-12 unfrozen layers. These results confirm that gradual unfreezing effectively leverages pre-trained representations for improved downstream performance.

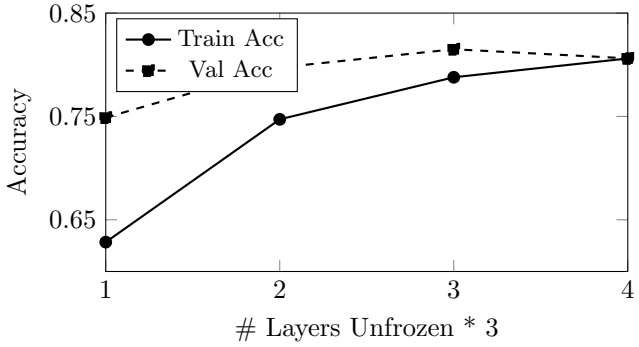

Figure 3: Training vs. validation accuracy as we gradually unfreeze 1–12 BERT layers.

4 Discussion

4.1 Challenges

Our re-implementation confronted some practical challenges. First, unlike the original work, we lacked access to the TRC2-Financial corpus for domain-specific pre-training. Consequently, we focused exclusively on the fine-tuning phase, which limited our capacity to shape the model's initial representations and made our results more sensitive to downstream hyperparameters.

Secondly, our first model and initial performance evaluation exhibited a puzzling disparity: while test-set accuracy reached 86%, computed precision, recall, and F_1–macro hovered around 35%. An audit of this issue revealed two main issues: (1) we had treated the Financial PhraseBank as binary when it actually contains three sentiment labels, and (2) we had not yet implemented k-fold cross-validation or class-weighted loss. We corrected this in our second iteration and the model produced results consistent with our expectations.

4.2 Interpretation of Results

With the corrected evaluation pipeline, our final FinBERT model obtained an average accuracy of 79.7% $\pm$ 2.8% and macro-F_1 of 62.5% $\pm$ 7.0% across 10 folds (Table 1). The gap between accuracy and F_1 reflects the class imbalance in the Financial PhraseBank (more positive than negative or neutral samples). Our use of a slanted triangular learning rate schedule, gradual unfreezing, and discriminative fine-tuning overall helped mitigate the issue of catastrophic forgetting and improved stability over training, as seen in the smooth loss and accuracy curves in Figures 2 and 3.

These results suggest that even without domain-specific pre-training, carefully tuned fine-tuning strategies can yield robust sentiment classifiers in low-resource settings. However, the relatively lower recall indicates that the model still struggles with underrepresented classes, motivating further balancing techniques.

4.3 Limitations

Our study was constrained by limited compute: on free-tier T4 GPUs, we could only run two main training iterations. This scarcity prevented extensive hyperparameter sweeps, each of our four major features (learning-rate schedule, unfreezing strategy, weight decay, class weighting) introduces multiple knobs, and searching for optimal solutions were infeasible given that colab would terminate our session after one k-fold CV loop. Additionally, omitting the TRC2-Financial corpus likely capped our ceiling performance, as the original authors reported modest but non-negligible gains from domain pre-training.

4.4 Future Work

To further improve performance, future efforts should:

- Incorporate domain-specific pre-training (e.g., on TRC2 or proprietary financial text) to better prime the model.
- Automate large-scale hyperparameter optimization (with Bayesian search for example) to fine-tune the slanted learning-rate ratio and layer-wise discriminative rates.
- Extend the model to regression tasks (e.g., sentiment scoring on the continuous FiQA dataset using a regularized MSE loss).

4.5 Practical Implications

Our fine-tuned FinBERT can rapidly classify and summarize incoming financial news streams, producing daily sentiment scores in seconds, an operation that might otherwise take an analyst hours. Integrating FinBERT with a lightweight web scraper could thus enable near-real-time sentiment dashboards for investors, enhancing decision support in fast-moving markets.

Overall, despite resource limitations, our re-implementation demonstrates that strategic fine-tuning methods can deliver a high-quality financial sentiment analyzer, laying a foundation for both academic exploration and real-world deployment.

References

[1] Doğu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv*, 2019.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019.

[3] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339. Association for Computational Linguistics, 2018.

[4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[5] Petri Takala. Financial PhraseBank Dataset. https://huggingface.co/datasets/takala/financial$_p hrasebank, 2014. Accessed : 2025 - 04 - 29.$